

WebNavi

DSL em português para validação de fluxos web

Paradigmas de Programação — Projeto Semestral

```
transicao de login para catalogo  
  via botao_entrar clicado  
  somente se campo_email preenchido  
fim
```

Ideia central

Transformar uma especificação legível em análise estática e testes automatizados.

Por que a WebNavi mudou?

A nova versão responde à crítica da primeira entrega.

Antes: risco de “autômato puro”

- Estados e transições existiam, mas com pouca semântica de página web.
- Faltava dizer onde a ação acontece: botão, campo, página, formulário.
- Parecia um grafo genérico, não um validador de fluxo web.

Agora: DSL de domínio web

- Páginas têm rotas e elementos declarados.
- Transições são disparadas por ações em elementos reais.
- Condições, efeitos, invariantes, dados e fluxos entram no modelo.
- A especificação pode gerar testes automatizados.

O que a WebNavi modela?

Uma navegação web descrita como estados, eventos e regras.

Página

estado do fluxo

/login,
/checkout

Elemento

objeto de interação

botão, campo,
aviso

Ação

evento do usuário

clicar,
preencher

Transição

mudança de tela

login →
catálogo

Condição

pré-requisito

campo_email
preenchido

Fluxo

cenário testável

compra
completa

```
pagina login
  rota /login
  elemento campo_email entrada obrigatorio formato email
  seletor "#email"
  elemento botao_entrar botao texto "Entrar" seletor
  "#btn-entrar"
fim
```

```
transicao de login para
  catalogo via botao_entrar
  clicado
  somente se campo_email preenchido
  entao sessao.autenticado recebe
  verdadeiro fim
```

3

Filosofia da sintaxe

Português, declarativa e legível

A sintaxe foi pensada para ser lida por quem define o fluxo, não só por quem programa.

Sem ;

fim de linha encerra declarações

Sem { }

blocos fecham com fim

Sem parênteses

expressões parecem frases

Sem operadores simbólicos

-> vira para, == vira igual a

Indentação significativa

define pertencimento

Palavras-chave em português

site, pagina, transicao, fluxo

Comparação

WebNavi anterior:

```
login -> dashboard on autenticar_ok;
```

WebNavi atual:

```
transicao de login para dashboard via  
  botao_entrar clicado  
  somente se campo_email preenchido  
fim
```

Como um arquivo .webnavi é organizado?

A especificação vira uma fonte única para documentação, validação e teste.

```
site loja

pagina login
  rota /login
  elemento campo_email entrada obrigatorio formato email
  seletor "#email"
  elemento campo_senha entrada obrigatorio formato senha
  seletor "#senha"
  elemento botao_entrar botao texto "Entrar" seletor
  "#btn-entrar" fim

inicio em inicio
final em confirmacao

transicao de login para
  catalogo via botao_entrar
  clicado
  somente se campo_email preenchido e campo_senha
  preenchido entao sessao.autenticado recebe verdadeiro
fim
```

1

site

nome do sistema
modelado

2

pagina

rotas e elementos de
interface

3

inicio/final

ponto de entrada e
estados finais

4

transição

navegação entre
páginas

5

invariante

propriedades que não
devem falhar

6

fluxo

sequências para gerar
testes

O que a implementação valida hoje?

A análise atual verifica consistência estrutural da especificação antes de executar o teste.

Páginas declaradas

início/final e transições apontam para páginas existentes

Elementos usados

botões e campos referenciados existem na página

Dados de teste

campos dos fixtures podem ser reutilizados nos fluxos

Fluxos

começam e terminam em páginas válidas

Cypress gerado

seletores e rotas viram comandos de teste

Validações profundas

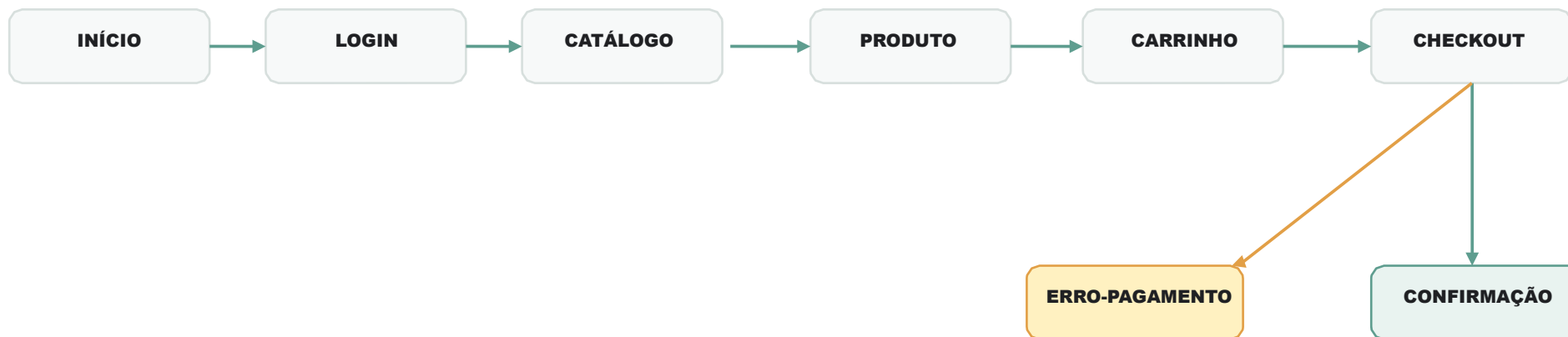
contradição lógica completa ainda é evolução

Loja virtual: compra completa

O exemplo demonstra um fluxo realista, com login, catálogo, carrinho, checkout e confirmação.

Fluxo principal

- Login com usuário válido
- Busca e visualização de produto
- Carrinho → checkout → pagamento



Caminhos alternativos

- Login inválido permanece em LOGIN
- Pagamento recusado vai para ERRO-PAGAMENTO
- Estoque indisponível pode bloquear compra

```
transicao de checkout para confirmacao  
via botao_pagar clicado  
somente se campo_cartao preenchido e campo_cvv preenchido  
entao pedido.numero recebe gerado  
fim
```

Da DSL para Cypress

O fluxo declarado é transformado em teste automatizado executável.

**Saída
gerada**

```
fluxo compra
  completa começar
  em inicio
  passo clicar botao_login aguardar pagina
  login passo preencher campo_email com
  usuario_valido.email
  passo preencher campo_senha
  com usuario_valido.senha
  passo clicar botao_entrar aguardar
  pagina catalogo
  passo clicar botao_pagar aguardar pagina
  confirmacao
  terminar em
  confirmacao fim
```

WebNavi

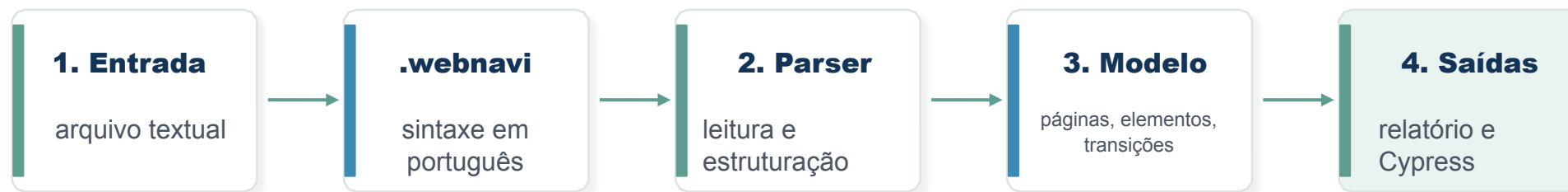


```
describe('fluxo: COMPRA-COMPLETA', ()
=> {
  it('deve navegar de INICIO até
  CONFIRMACAO', () => {
    cy.visit('/');
    cy.get('#login').click();
    cy.url().should('include',
    '/login');
    cy.get('#email').type('usuario
    @teste.com');
    cy.get('#senha').type('Senha@1
    23');
    cy.get('#btn-entrar').click();
    cy.url().should('include',
    '/produtos');
    cy.get('#pagar').click();
    cy.url().should('include',
    '/confirmacao');
  });
});
```

Cypress

Pipeline da implementação

Entrada WebNavi → modelo interno → validação → relatório/teste.



Implementação atual

Protótipo em notebook/código: estruturas de dados, parser simples, análise estática e geração de teste.

Importante

A gramática formal descreve a linguagem ideal; o protótipo implementa o subconjunto demonstrado.

Evolução planejada

Caminhos claros para transformar o protótipo em uma DSL mais robusta.

Parser formal

ANTLR, Lark ou combinadores para implementar melhor a gramática.

Análise semântica

alcançabilidade, estados presos, referências inválidas e condições conflitantes.

Invariantes fortes

verificar regras globais sobre o grafo de navegação.

Cobertura de fluxos

indicar quais páginas/transições não foram cobertas por nenhum fluxo.

Geração ampliada

suporte a Playwright, fixtures, asserts e cenários negativos.

Editor/validador

feedback visual para escrever arquivos .webnavi com menos erro.